

Real-Time Semantic Segmentation of Spherical Images for Automotive Applications

Anthony Dell'Eva (✉ anthony.delleva2@unibo.it)

University of Bologna

Marco Orsingher

University of Parma

Pietro Cerri

Ambarella (United States)

Massimo Bertozzi

University of Parma

Stefano Ghidoni

University of Padua

Francesca Ghidini

Ambarella (United States)

Research Article

Keywords: Real-Time Semantic Segmentation, Spherical Images, Spherical Convolutions, Autonomous Driving

Posted Date: July 4th, 2023

DOI: <https://doi.org/10.21203/rs.3.rs-3123233/v1>

License: © ⓘ This work is licensed under a Creative Commons Attribution 4.0 International License.

[Read Full License](#)

Additional Declarations: No competing interests reported.

Real-Time Semantic Segmentation of Spherical Images for Automotive Applications

Anthony Dell'Eva^{1,3*}, Marco Orsingher^{2,3}, Pietro Cerri³, Massimo Bertozzi²,
Stefano Ghidoni⁴, Francesca Ghidini³

¹University of Bologna, Bologna, Italy.

²University of Parma, Parma, Italy.

³VisLab - Ambarella, Parma, Italy.

⁴University of Padova, Padova, Italy.

*Corresponding author(s). E-mail(s): anthony.delleva2@unibo.it;

Contributing authors: marco.orsingher@unipr.it; pcerri@ambarella.com;

bertozzi@ce.unipr.it; stefano.ghidoni@unipd.it; fghidini@ambarella.com;

Abstract

Recent advancements in autonomous driving technology have resulted in a growing need for robust algorithms that can effectively detect, recognize, and segment objects in the surrounding environment. Semantic segmentation systems, which classify each pixel of an image into a class, have become a key component in achieving complete scene understanding, a crucial task for self-driving vehicles. This work focuses on utilizing deep learning methods to perform semantic segmentation on spherical images, a particular representation of fisheye images, characterized by a large field of view and strong distortion, which can result in performance degradation on convolutional neural networks (CNNs) trained on pinhole images. To address this issue, we propose the use of spherical kernels, which adapt their sampling locations according to the spherical projection, as an alternative to standard convolutional kernels, in real-time CNNs. We demonstrate that this approach outperforms the use of the regular convolution operation. To showcase the effectiveness of this method, we generate three spherical image datasets with varying characteristics.

Keywords: Real-Time Semantic Segmentation, Spherical Images, Spherical Convolutions, Autonomous Driving

1 Introduction

In recent years, the field of self-driving technologies has experienced an exponential growth. This growth has been accompanied by an increasing

interest trend from industries and research and development (R&D) departments, which has led to a greater attention by institutions and the media. The role of institutions is particularly

important due to the current lack of government laws and regulations in the field, which are hampering the evolution of the technology rather than encouraging development and testing. Similarly, the media plays a key role in raising awareness about autonomous vehicles (AVs) among the general public, and in helping people to become familiar with the implications this revolution could bring on different levels, from mobility efficiency and road safety, with a consequent huge number of saved lives, to increased productivity and new business models [1].

In this work, we address the semantic segmentation problem in automotive scenarios, which involves classifying each pixel of an image with a particular label such as cars, pedestrians and poles. An accurate solution to this high-level task enables a better comprehension of the surrounding environment and lays the foundation to complete scene understanding. In particular, our work focuses on the segmentation of spherical images, a particular representation of fisheye images that is characterized by a wide field of view and, consequently, a considerable distortion. This type of image is exploited in short-range systems of self-driving cars to obtain a 360° monitoring of the vehicle.

Semantic segmentation problems are commonly tackled using deep learning models, particularly convolutional neural networks and transformers [2–4], due to their superior accuracy and efficiency compared to other classical computer vision methods. Although transformer-based models have shown promising results, CNNs remain the most efficient for real-time deployment. However, supervised training of these models requires a substantial amount of labeled data. The lack of public annotated datasets consisting of spherical images has been addressed with the creation of three different datasets, converting geometrically a well-known perspective images dataset, generating synthetic images with an urban scenes simulator and hand-annotating real spherical images. The image distortion caused by the spherical projection, instead, has been handled by modifying the CNNs architectures with the application of spherical convolutional kernels, whose sampling patterns are adapted to follow the same distortion introduced by the spherical model, encoding invariance to it explicitly in the neural network. Previous studies on semantic segmentation of images with equirectangular distortion focus only on 360-degree panoramic images [5–8]. In addition, they do not take into account any inference time constraint. Differently, our work investigates the semantic segmentation of reprojected fisheye images using real-time CNNs, a crucial requirement in the self-driving vehicles field, on both

synthetic and real spherical image datasets. Our study extends the existing literature by demonstrating the effectiveness of the approach under practical conditions in the context of autonomous driving.

In summary, our contributions are:

- we apply spherical convolutional kernels on two different real-time CNNs to perform semantic segmentation on spherical images in autonomous driving scenarios;
- we show that CNNs with spherical convolutional layers outperform the same CNNs trained with standard convolutions on three distinct spherical image datasets;
- we demonstrate that models trained on perspective images can be utilized for semantic segmentation on spherical images by simply modifying the sampling locations of the learned kernels.

This article is organized as follows: in Section 2 we review related works. In Section 3 we describe our method, including the generation of spherical datasets and the modification of CNN architectures. Section 4 shows the experimental results, while Section 5 concludes the paper.

2 Related work

Different problems of recent interest, such as omnidirectional vision for drones, robots, self-driving cars and virtual reality applications, have

created a need for models that can work on spherical images. However, standard CNNs suffer from significant performance degradation on these images. This is mainly due to the fact that the natural projection surface for spherical images is a sphere, which cannot be unwrapped to a plane without introducing considerable distortion, particularly in the polar regions.

Geometrical Priors

The promising results showed by incorporating geometrical prior knowledge into neural networks have recently attracted more and more attention [9, 10]. Dai et al. [11] introduce two new modules to improve the modeling capability of CNNs, namely deformable convolution and deformable RoI pooling. The idea is to augment the spatial sampling locations in the modules offsets that can be learned from the target task without additional supervision. The modules can easily replace their counterparts in regular CNNs and can be trained end-to-end with standard back-propagation. Zhu et al. [12] notice that Deformable CNNs can cause features to be influenced by irrelevant image content. Therefore, they present a reformulation of the deformable convolution that enhances the ability to focus on pertinent RoI. A further modification is introduced by [13], with a restricted deformable convolution which employs a fixed central sampling location, showing that the framework is able to handle effectively fisheye images. A

similar approach is presented by Su et al. [14–16], whose Kernel Transformer Network (KTN) learns a function that generates specific kernels for different distortions, taking as input a source kernel and spherical coordinates. This allows model transfer: the same model can be applied to different source CNNs with the same base architecture. In [17], the theoretical building blocks for constructing spherical CNNs are introduced. The authors propose a spherical cross-correlation that is both expressive and rotation-equivariant, and can be computed efficiently using a generalized FFT algorithm. [18] performs object detection on panoramic images, extending the previous work with a novel grid-based spherical convolution that avoids the loss of the object location in the image. SpherePHD [19] utilizes a spherical polyhedron to represent omnidirectional views, minimizing the variance of the spatial resolving power on the sphere surface. New triangular-shaped convolution and pooling modules to be applied to the proposed representation are included.

Recent works [20–22] propose different frameworks to perform convolution directly on an accurate discretization of the sphere, namely icosahedral spherical mesh. Exploiting this approximation, [23] renders spherical images to a set of locally-planar image grids tangent to it in order to train standard CNNs, obtaining results comparable to many specialized spherical kernels.

Eder et al. [24] propose another representation for spherical images based on the icosahedral Snyder equal-area projection (ISEA), a projection onto a geodesic grid which is nearly conformal, nearly equal-area and nearly equidistant. The network kernels are modified projecting the planar filters onto the icosphere surface. An application of 3D spherical convolutions to predict proteins structure is proposed in [25], where two strategies are presented, using either a spherical-polar grid or a grid based on the cubed-sphere representation of the sphere.

Coors et al. [26] and Tateno et al. [27] developed a location-dependent transformation of the regular convolutional kernels using a gnomonic projection to adapt to the spatial distortion of the spherical image. The key concept behind the distortion-aware convolution is that the sampling grid is deformed according to the image projection model, so that the receptive field is rectified, encoding invariance against such distortions explicitly into the neural network. A similar idea is exploited in [28] to perform 3D layout estimation. Equivariant maps for Platonic pixelizations of the sphere, which have efficient implementations, are combined with an equivariant padding procedure in [29] to build deep equivariant networks.

Panoramic Semantic Segmentation

The semantic segmentation of 360° panoramic images is performed in [30, 31] where data augmentation and other expedients such as padding and dynamic loss weights are employed to face the border discontinuity and the classes unbalance. To address the distortion problem, [32] transforms panoramic images to perspective images before the segmentation process, while [5, 7] adopt equirectangular convolutions in a modified version of standard segmentation networks. The benefit of the spherical representation of planar images for the semantic segmentation task with high-resolution datasets is investigated by Zhang et al. [33]. Specifically, they propose to employ spherical CNNs to reduce distortions otherwise introduced by the image plane. In [34] a concurrent horizontal and vertical attention module to leverage width-wise and height-wise contextual priors available in the panoramas is introduced. To yield semantic segmenters suitable for wide FoV images, they present a multi-source omni-supervised learning scheme with panoramic domain covered in the training via data distillation. [35] presents a novel method to transform the spherical signals to structured vectors that can be processed through standard CNNs directly. The distortion convolutional module (DCM) is introduced in the network encoder by Hu et al. [36]. The module is developed to correct the

image distortion according to the image-forming principle. To overcome the lack of annotated panoramic images, [37] proposes a framework which allows model training on standard pinhole images and transfers the learned features to the panoramic domain. DensePASS [38] formalizes the problem of unsupervised domain adaptation for panoramic segmentation. The source domain dataset contains pinhole images with semantic annotations, while the target dataset contains panoramic images without annotations. A similar adaptation approach is employed by [8], where the Deformable Patch Embedding (DPE) and the Deformable MLP (DMLP) modules endow the network with distortion awareness.

Panoramic Datasets

Commonly used datasets for semantic segmentation contain images acquired with standard FOV cameras [39, 40]. A synthetic panoramic dataset generated by stitching different images from SYNTHIA sequences dataset is presented in [6]. [7] releases the first pixel-level annotated outdoor panoramic image dataset (CVRG-Pano), consisting of 600 images belonging to the urban and suburban parts of Pittsburgh City. [34] introduces Wild PANoramic Semantic Segmentation (WildPASS) dataset which gathers panoramas of 25 cities from all continents. All images are finely labeled for semantic segmentation of 6 classes. The OmniScape dataset [41] includes synthetic stereo

256 fisheye and catadioptric images acquired from the
 257 two front sides of a motorcycle, including seman-
 258 tic segmentation, depth map, intrinsic parameters
 259 of the cameras and the dynamic parameters of the
 260 motorcycle. The images are generated from GTA
 261 V and CARLA simulators. Despite the numerous
 262 datasets comprising panoramic images presented,
 263 their suitability in this specific case is limited due
 264 to our emphasis on spherical images.

272 3 Method

273 Semantic segmentation of spherical images and, in
 274 general, of images with wide field-of-views (FOVs)
 275 and strong distortion, pose significant challenges
 276 due to the lack of large-scale public datasets, par-
 277 ticularly in the context of automotive scenarios. In
 278 this section, we describe our approach to address
 279 these issues.

287 3.1 Datasets creation

288 To overcome the challenge of the lack of large-
 289 scale datasets composed of spherical images, we
 290 generated three different datasets to validate our
 291 approach. To this end, we modified Cityscapes
 292 [39], a well-known urban scenes dataset used as a
 293 benchmark for evaluating semantic segmentation
 294 approaches. We tailored Cityscapes to our needs
 295 by means of a geometrical mapping to obtain
 296 spherical images without human effort, similar to
 297 what has been done for fisheye images in previous

studies [42, 43]. The dataset consists of 5000 pixel-
 level annotated images manually selected from
 27 cities, taken at different daytime and weather
 conditions to capture diverse foreground objects,
 backgrounds, and overall scene layout. It provides
 labels for 19 classes, and it is split into three sub-
 sets: 2975 images for training, 500 for validation,
 and 1525 for testing. The test annotations are not
 publicly available, and hence, all the experimental
 results presented in Section 4 are compared with
 respect to the performances on the validation set.
 We created a mapping look-up table (LUT) that
 associates the source pinhole image coordinates
 to the target spherical image coordinates. Each
 LUT element is computed by projecting a pinhole
 image point, represented in the uv image reference
 system, to the xyz camera reference system using
 inverse perspective projection equations. The new
 coordinates are projected back to the uv coordi-
 nate using the spherical model. In the practical
 implementation, the computation is performed by
 swapping the source and target image (i.e., from
 spherical to pinhole) to obtain a dense result. A
 training image is shown in Figure 1. As we will
 see in Section 4, even if the small FOVs presented
 by pinhole images (i.e., 50° horizontally and 25°
 vertically) allowed us to obtain only a slight distor-
 tion, the validation of our approach has not been
 significantly affected.

To address the FoV issue, we examined the
 possibility of generating a dataset comprising

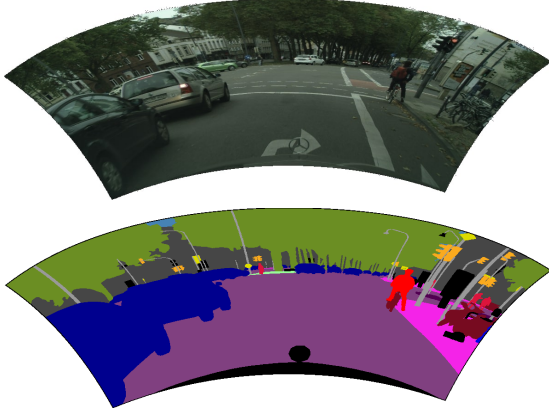


Fig. 1 Examples of training image and associated annotation resulting from the geometrical mapping, derived applying a $1rad$ pitch rotation to the spherical camera in order to obtain a stronger distortion.

spherical-like images obtained from a software simulator. This approach was aimed at providing more compelling evidence of the benefits associated with the incorporation of a spherical convolutional layer into the network architecture. Our objective was to identify a simulator capable of replicating synthetic urban traffic scenarios while enabling us to manipulate the virtual camera lens or the corresponding FoVs. In the former case, we would generate a fisheye image dataset that later would be converted into a spherical image one. In the latter case, we would acquire pinhole images with FoVs comparable to real fisheye lenses and subsequently would use the same geometric mapping approach utilized to build the spherical Cityscapes dataset. To determine the most suitable urban scene simulator, we analyzed the pros and cons of different options, including *Airsim* [44], *LGSVL* [45], *rFpro* [46], *CARLA* [47], taking

into account factors such as image quality, scenario diversity, and the availability of semantic labels.

Despite its inferior image quality, the *CARLA* simulator provided the best trade-off between the aforementioned factors. We obtained the frames using a virtual pinhole camera with horizontal and vertical FoVs of, respectively, 150° and 140° for the first dataset and 160° and 150° for the second one. The only difference between the two datasets was the removal of motion blur in the second set to obtain sharper images. We refer to the two image collections by their respective horizontal FoVs. The 150° set consists of a total of 2394 frames, divided into 1505 for training, acquired from three different maps, 496 for validation, and 393 for testing, obtained from two additional scenarios. The second set is composed of 2264 images, of which 1436 are used for training, 396 for validation, and 432 for testing. All the frames were captured with random cloudiness, precipitation, and sun altitude angle parameters to provide a variety of weather conditions and daytime. The semantic classes included 12 urban scene categories such as *car*, *road*, *pole* and *traffic sign*. *Sky* and *terrain* form a single class since attempts to generate the ground truth by segmenting them by color and position failed due to illumination changes and the presence of *terrain* above the horizon line. An example of a training sample with its semantic labels is shown in Figure 2. Note that the

derived images have a circular shape, with the corners excluded from the spherical projection. This was due to the limited FoVs set for the virtual pinhole camera: to generate rectangular-shaped images, we would require a 180° FoV, which would result in an infinite dimension pinhole image.

In the third case, we created a hand-annotated dataset, despite its considerable time-consuming nature. The annotated spherical images were generated using a simple tool that allows for the drawing of polygons around objects of interest and the assignment of specific labels, accordingly. The creation of a large dataset with pixel-wise semantic annotations is known to require significant human effort to mark instance boundaries accurately. Therefore, we limited the labeling to seven classes, taking advantage of semantic similarity to group different objects into the same category when possible. In particular, the categories include *free space*, *sidewalk*, *poles*, *two-wheeled vehicles*, *vehicles*, *pedestrians*, and *background*. The dataset comprises a total of 1403 spherical images, divided into 1127 for training, 192 for validation, and

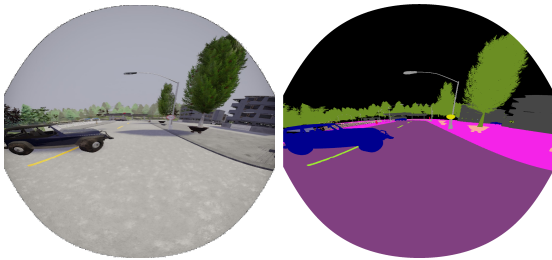


Fig. 2 Example of training image and corresponding annotation of our synthetic spherical dataset generated with CARLA simulator.

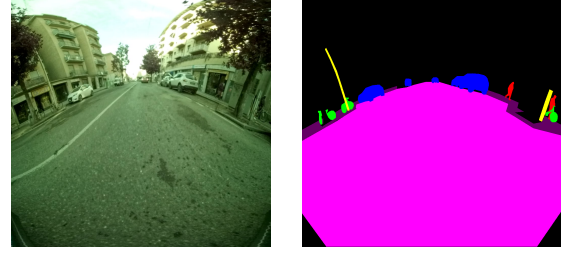


Fig. 3 Training image and corresponding annotation extracted from our hand-labeled spherical dataset.

84 for testing. The majority of the images were captured under favorable weather conditions (i.e., sunny days), while only a small portion was taken under challenging illumination conditions (i.e., rainy or snowy days, night lighting, and underground parking). An example of a training image and its respective annotation is depicted in Figure 3.

3.2 Spherical convolutional kernels

To mitigate the distortion issue, we opted not to rely on specific techniques to train the deep learning model, but instead proposed a revision of the model architecture itself. Specifically, we replaced the standard convolutional layers with spherical kernels that adapt their sampling locations to the characteristic distortion of the spherical projection.

The proposed framework, which encodes invariance against distortions directly into CNNs, is inspired by Coors et al. [26]. They applied the model to 360° omnidirectional images and demonstrated its superiority over regular CNN baselines

in both image classification and object recognition. Our objective is to showcase its benefits when integrated into model architectures specifically designed for real-time semantic segmentation tasks.

In the upcoming description of the convolutional operator and its corresponding modification, the notation will only refer to the two spatial dimensions, omitting the additional dimensions related to batch size and number of channels. The 2D convolution operation is computed by sampling the input feature map f_l at layer l with a regular grid R . For a standard 3×3 filter, R is defined as follows:

$$R = (-1, -1), (-1, 0), \dots, (1, 0), (1, 1). \quad (1)$$

Next, the weighted summation of the sampled values using the convolutional kernel k is performed. For each 2D spatial location $p = (x_p, y_p)$ on the input feature map, the elements of the output feature map f_{l+1} are calculated as follows:

$$f_{l+1}(p) = \sum_{r \in R} k(r) f_l(p + r) \quad (2)$$

where r denotes the pixel relative location in R . In the spherical convolution, a function $\gamma(p, r)$ computes a new distorted sampling pattern according to the image projection model, altering the sampling grid R . Therefore, equation 2 is modified

to:

$$f_{l+1}(p) = \sum_{r \in R} k(r) f_l(p + \gamma(p, r)) \quad (3)$$

This sampling grid deformation enables the rectification of the receptive field. Since the sampling locations computed with $\gamma(p, r)$ are real values, the corresponding feature maps values are computed by interpolation. Specifically, the value of the feature map at fractional pixel location $\tilde{p} = p + \gamma(p, r)$ is obtained as a weighted sum of the values at the four integer sampling points adjacent to $\tilde{p}$, denoted by $N(\tilde{p})$, using an interpolation kernel $G(\cdot, \cdot)$. Mathematically, we have:

$$f_l(\tilde{p}) = \sum_{q \in N(\tilde{p})} G(q, \tilde{p}) f_l(q) \quad (4)$$

In the case of bilinear interpolation, the kernel is defined as:

$$G(q, \tilde{p}) = \max(0, 1 - |x_q - x_{\tilde{p}}|) \max(0, 1 - |y_q - y_{\tilde{p}}|) \quad (5)$$

where x_q and y_q denote the coordinates of the point q , while $x_{\tilde{p}}$ and $y_{\tilde{p}}$ denote the coordinates of the fractional location $\tilde{p}$. On the other hand, in the case of nearest neighbor interpolation, the kernel is defined as:

$$G(q, \tilde{p}) = \delta(\lfloor x_q + 0.5 \rfloor - x_{\tilde{p}}) \delta(\lfloor y_q + 0.5 \rfloor - y_{\tilde{p}}) \quad (6)$$

where $\delta(\cdot)$ denotes the Kronecker delta function.

In order to obtain the distorted pixel location $\gamma(p, r)$ and the relative location $r = (x_r, y_r) \in R$ of the new sampling grid, the image (or feature map) point $p = (x_p, y_p)$ is first transformed into spherical coordinates, resulting in the point $p_s = (\theta, \phi)$. This is achieved by computing the longitude θ and the latitude ϕ as:

$$\theta = \left(x_p - \frac{w}{2}\right) \frac{FoV_h}{w} \quad (7)$$

$$\phi = \left(\frac{h}{2} - y_p\right) \frac{FoV_v}{h} \quad (8)$$

Here, w and h are the width and height of the input in pixels, respectively, and FoV_h and FoV_v represent the horizontal and vertical field of view.

Next, the longitude and latitude θ, ϕ are mapped onto a unit sphere coordinate system, obtaining the point $p_u = (x_u, y_u, z_u)$. The tangent plane $T_u = (t_x, t_y)$ of the unit sphere at point p_u is then computed as follows:

$$t_x = v \times p_u \quad (9)$$

$$t_y = p_u \times t_x \quad (10)$$

where $\times$ denotes the cross product and $v = (0, 1, 0)$ corresponds to the upper vector of the unit sphere coordinate system.

Now, the distorted pixel locations grid on the input image can be obtained by back-projecting a regular grid sampling pattern on the tangent plane T_u to the image coordinate system. The sampling

grid on the tangent plane is computed using the two axes t_x and t_y and the relative element $r \in R$ of the original grid as:

$$r_{sph} = (\rho_u(t_x \cdot r_x), \rho_u(t_y \cdot r_y)) \quad (11)$$

where $\rho_u = \tan \frac{FoV_h}{w}$ denotes the spatial resolution on the unit sphere coordinate system and $\cdot$ represents the scalar product.

Each location on the sampling grid related to the tangent plane can be calculated as:

$$\tilde{p}_u = p_u + r_{sph} \cdot \quad (12)$$

The distorted grid is then obtained by back-projecting each element $\tilde{p}_u$ to spherical coordinates using the inverse equations of the spherical model. The longitude $\theta_{\tilde{p}}$ and latitude $\phi_{\tilde{p}}$ are converted into image coordinates as follows:

$$x_{\tilde{p}} = \left(\frac{\theta_{\tilde{p}}}{FoV_h} + \frac{1}{2}\right) w \quad (13)$$

$$y_{\tilde{p}} = \left(\frac{1}{2} - \frac{\phi_{\tilde{p}}}{FoV_v}\right) h \quad (14)$$

The previously introduced function $\gamma(p, r)$ returns the coordinates offsets $(x_{\tilde{p}} - x_p, y_{\tilde{p}} - y_p)$, which are independent of the input image and only depend on the distortion model. Therefore, they do not require recomputation for each new input image. For spherical images, the distortion remains constant for the same horizontal location,

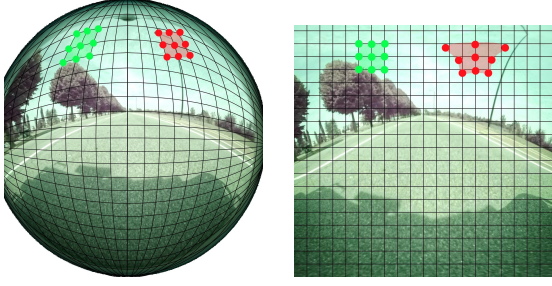


Fig. 4 Visualization on a fisheye image (left) and on its spherical representation (right) of the sampling patterns of a regular convolutional kernel, in green, and its spherical adaptation, in red.

and thus, only a set of $h|R|$ offsets needs to be stored, where $|R|$ is the number of elements in the sampling grid.

The sampling pattern of the new convolutional kernel is distorted in the same way as objects are distorted when projected onto a spherical image representation. The proposed approach is illustrated in Figure 4 provides a visualization of the idea. The application of these distortion-aware convolutional kernels to deep learning models allows to retain the original spherical image connectivity throughout the different network layers, rectifying the receptive fields. In the next section, we will delve into one of the major issues that we investigated, which is the different improvement or degradation of model performance as a function of the number and position of the implemented spherical convolutions.

3.3 CNNs architecture

The proposed methods were validated using two real-time convolutional neural networks, namely

ERFNet [48] and HarDNet [49]. Note that other well-known semantic segmentation models, such as DeepLab [50] and Mask2Former [4], may have achieved better results. However, for autonomous driving applications, a network that does not meet real-time requirements is impractical.

ERFNet provides a good trade-off between high-quality semantic segmentation and computational resources, making it an efficient architecture. This efficiency is attributed to a redesigned layer that uses residual connections and factorized convolutions to achieve both accuracy and efficiency. Romera et al. [48] introduced the redesign of the non-bottleneck residual module originally proposed in [51], utilizing only convolutions with 1D kernels. As demonstrated in [52], any 2D filter can be decomposed into a combination of 1D filters. Specifically, the weights of a 2D convolution $W_{2D} \in \mathbb{R}^{C \times d \times d \times F}$ can be represented as a pair of $W_{1D} \in \mathbb{R}^{C \times d \times F}$, with C , d , F being the number of input channels, the kernel size, and the number of output channels (feature maps), respectively. Leveraging this factorization, the residual bottleneck module's 3×3 convolutions were modified to propose a faster module with equivalent learning capacity. This new module, called non-bottleneck 1D (*non-bt-1D*), resulted in a 33% reduction in parameters compared to the original module, increasing computational efficiency. The encoder-decoder architecture, which provides a comprehensive overview, is summarized in Table

A1. Please refer to the appendix for the detailed table.

In the experiments, the modification of the module was performed by substituting the first two 3×1 and 1×3 convolutional filters with a single 3×3 spherical kernel, increasing the number of network parameters. To demonstrate the benefits of the new kernel’s application to existing models modifying only how they operate from a geometrical point of view, it is important to maintain a comparable number of parameters with respect to the original version. As described in the evaluation section, the best results are achieved by altering only one layer of the network. In this particular case, the number of additional parameters introduced is entirely negligible: this ensures a fair comparison between the modified models and their original counterparts, highlighting the specific benefits brought about by the new kernel. Note that since the *Downsampler block* consists of a concatenation of a convolutional operation and a max pooling operation, the latter is also modified to adhere to the spherical sampling pattern. In addition to leveraging the benefits of the new module, the memory and computation requirements are further reduced through careful observations and design choices. Firstly, the encoder of the proposed model follows a sequential structure, avoiding the fusion of feature maps from different layers to obtain a fine-grained

output, as seen in the Fully Convolutional Network (FCN) [53]. Secondly, the model does not incorporate long-range skip connections between the encoder and decoder. These connections, as implemented in previous works [54], are omitted. Lastly, unlike architectures such as SegNet or ENet [55, 56], the decoder solely employs deconvolution layers for upsampling, without resorting to max unpooling operations. This design choice eliminates the necessity to share pooling indexes from the encoder.

The HarDNet architecture aims to optimize memory traffic for accessing intermediate feature maps. Chao et al. [49] have demonstrated that predicting the inference time solely based on model size is not accurate, and that memory traffic can be the dominant factor in determining model latency. To improve efficiency, they proposed a novel building block, called the Harmonic Dense Block (HDB), which is a redesign of the building block in DenseNet [57]. The new connection pattern of the HDB creates a block that resembles an overlapping of harmonic waves, hence the name Harmonic Densely Connected Network (HarDNet). Table A2 in the appendix provides a summary of the encoder-decoder structure of the network. Additionally, a brief comparison of HarDNet with ERFNet is presented in Table 1.

Architecture	#Params (M)	Mean Inference Time (ms)	FPS	Mem. Occupancy (MiB)
ERFNet	2.2	18	56	653
HarDNet	4.1	23	43	677

Table 1 ERFNet and HarDNet comparison. The values are referred to inferences performed on 576×640 images using a Titan Xp GPU. We can notice how, despite having approximately twice the number of parameters compared to ERFNet, HarDNet achieves real-time inference.

4 Experimental evaluation

This section presents the experimental results obtained by leveraging the distortion-aware convolutional kernel on various spherical datasets. The primary objective is to demonstrate the benefits of the filter applied to the semantic segmentation task. To this aim, we first focused on the advantages brought by training the models directly with the modified architecture. Secondly, we studied the capability of leveraging CNNs trained on pin-hole images, introducing the spherical layers in the model architecture only for inference on spherical images.

4.1 Setup

ERFNet and HarDNet models are implemented in Python 3.7 with the support of PyTorch 1.1 framework and are trained using an NVIDIA Titan Xp GPU with 12 GB of GDDR5 VRAM and CUDA 10.2 toolkit.

Training is performed employing the Adam optimization of stochastic gradient descent [58]. After an initial tuning, we set a batch size of 8, weight decay (L2 penalty) of $1e^{-4}$, momentum of 0.9 and a starting learning rate of $5e^{-4}$ which follows a polynomial decay policy, i.e. $lr =$

$lr_{init}(1 - epoch/max_epochs)^{0.9}$. The number of epochs is set to 500 for ERFNet and 700 for HarDNet, which showed a slower convergence. However, early stopping is defined to end the training if the performance on the validation set does not improve for more than 50 consecutive epochs. The employed loss function is the weighted cross-entropy.

The accuracy of the algorithms is evaluated using the Jaccard index, also known as Intersection over Union (IoU). The function is computed for each specific class as follows:

$$IoU = \frac{TP}{TP + FP + FN} \quad (15)$$

where TP, FP and FN are the number of true positives, false positives and false negatives at pixel level, respectively. In order to compare the different results, we calculated the average between the classes, obtaining the mean IoU (mIoU).

We applied different initialization and input dimensions. For the spherical Cityscapes dataset, we employed the best model obtained training on original pinhole Cityscapes and inputs of 368×1024 . For the CARLA dataset, we used the parameters trained on ImageNet and 536×576 inputs.

Finally, for our hand-labeled dataset, we used the weights of the network trained on pinhole Cityscapes only for the available 7 classes instead of the original 19, and 496×512 inputs.

4.2 Training with spherical convolution

The impact of employing spherical kernels for semantic segmentation on the spherical Cityscapes dataset is presented in Table 2. The direct incorporation of the projection model distortion into the network structure resulted in a relatively marginal but noteworthy improvement. The Cityscapes conversion provided partial evidence regarding the efficacy of applying spherical kernels in the CNN convolutional layers. The limitation lies in the constrained horizontal and vertical Fields of View (FoVs), which are determined by the lens focal length and camera sensor size, and cannot be augmented through post-processing algorithms. To elaborate on this point, Figure 5 illustrates a comparison between the FoVs of a typical spherical image and the FoVs of our image obtained from a pinhole camera. Notably, superior results are achieved only when the new convolution is applied to the initial layer of the encoder. Intuitively, the input requires rectification only once, after which regular convolutions can be applied to the output feature maps. On the contrary, the implementation of multiple consecutive spherical convolutions

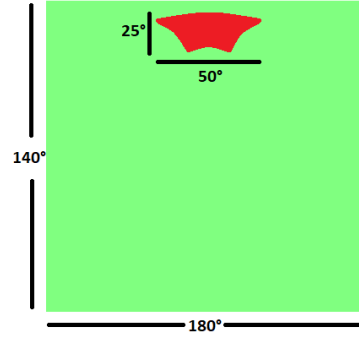


Fig. 5 Comparison of the horizontal and vertical FoVs between a spherical image obtained from a real fisheye image, in green, and our spherical image derived from the back-projection of a pinhole image with a $1rad$ pitch rotation, in red.

cannot be explained geometrically and, as a result, applying the new filters to every encoder layer led to a performance degradation.

The results referred to the synthetic datasets generated with the CARLA simulator are presented in Table 3. The %mIoU is computed on the test set, which is available in this case. The efficacy of spherical convolution is confirmed when applied to synthetic images. It is observed that the improvement is more pronounced, likely due to the wider horizontal and vertical Fields of View (FoVs). Interestingly, contrary to our expectations, the dataset with a horizontal FoV of 160° did not exhibit a larger improvement compared to the one with $FoV_h = 150^\circ$, despite the greater distortion. It is important to note that both datasets are relatively small, comprising only about 2300 images, and the test images are generated from a single scenario with limited object variety, which differs between the two datasets.

While the observed results may not demonstrate significant generalization performance, they are still crucial for the specific aspects we aim to demonstrate in our study.

Table 4 shows the final %mIoU on the test set of our manually annotated dataset. As can be observed from the results, the employment of spherical convolutions enhanced the final metric for both segmentation models. In order to highlight the improvements achieved through the utilization of spherical convolution kernels, it is worth noticing the limitations of the dataset under consideration. Firstly, the dataset is small and exhibits a high class imbalance, with only a few examples of *two-wheeled vehicles* and *pedestrians* in the training set. Secondly, the *background* label encompasses a wide intraclass variety, including different types of objects, resulting in potential interclass similarities, such as tree trunks annotated as *background* and lampposts labeled as *pole*. Despite the utilization of class weighting techniques, obtaining satisfactory results for less frequent classes proved challenging. It is plausible that with a larger dataset and a greater number of labeled classes, the benefits of distortion-aware convolution would become even more evident.

The supplementary video (Online Resource 1) presents the qualitative results obtained by the proposed method. In the provided real driving sequence recorded in Italy, our approach exhibits the capability to accurately segment all seven

classes, with a particular emphasis on vulnerable road users, namely pedestrians and cyclists. Notably, the clip effectively demonstrates the ability of our method to address distortions and produce satisfactory segmentation results for categories that are mostly affected by this kind of projection, such as poles and sidewalks.

4.3 Leveraging models trained on pinhole images

Following the analysis of the benefits obtained from training CNNs with modified convolutions, we explored the potential of exploiting network parameters from models previously trained on pinhole images. In this case, we did not utilize the weights for network initialization; instead, we directly employed them for inference on spherical images by replacing only the model’s regular convolutional kernels with distortion-aware ones. It is crucial to note that modifying the filters sampling pattern does not alter the number of network parameters.

For our experiments, we opted to replace only the first regular convolutional layer, as it yielded the most significant benefits. Table 5 showcases the quantitative results obtained from this inference strategy on spherical Cityscapes. The parameters utilized correspond to the best model trained on pinhole Cityscapes images. The degradation in overall performance can likely be attributed to the

Model	Spherical conv.	Layers/block modified	% mIoU	$\Delta\%$
ERFNet	No	-	63.12	-
ERFNet	Yes	1	64.25	+1.13
ERFNet	Yes	1-16	57.21	-5.91
ERFNet	Yes	13-16	62.33	-0.79
HarDNet	No	-	62.92	-
HarDNet	Yes	I (1 st conv)	63.41	+0.49

Table 2 Quantitative results for the spherical Cityscapes dataset. The symbol $\Delta\%$ represents the percentage difference in mIoU compared to the original architecture with regular convolutions. For detailed information on the layers/block numbers, please refer to Tables A1 and A2 in Appendix A.

FoV_h	Model	Spherical conv.	Layers/block modified	% mIoU	$\Delta\%$
150	ERFNet	No	-	45.85	-
	ERFNet	Yes	1	50.33	+4.48
	ERFNet	Yes	16	50.91	+5.06
	ERFNet	Yes	13-16	46.81	-0.96
	ERFNet	Yes	9-16	44.13	-1.72
160	ERFNet	No	-	46.54	-
	ERFNet	Yes	1	48.35	+1.81

Table 3 Quantitative results for the CARLA synthetic datasets, where $\Delta\%$ denotes the percentage change in mIoU compared to the original architecture with regular convolutions. Please refer to Tables A1 and A2 in Appendix A for details regarding the layers/block numbers.

white bands present at the image edges, which are ignored during IoU computation but negatively impact inference at the borders. Moreover, the positional bias of the weights can lead to incorrect predictions due to the displacement of objects within the scene caused by the spherical transformation. Despite these limitations, we can notice a discernible relative improvement when comparing our model incorporating spherical convolutions to the original model with standard convolution.

Table 6 illustrates the results obtained from applying this strategy to our hand-labeled dataset. To address the lack of annotated pinhole images, since the domain shift between our dataset and Cityscapes makes the latter’s images unsuitable for our problem, we trained the models on small

patches obtained by cropping the central region of the original spherical images. Indeed, the central region exhibits less pronounced distortion, allowing us to approximate the resulting images as pinhole. For subsequent experiments, these patches were geometrically transformed into pinhole images. The points within the spherical image coordinates of the patches were converted into camera coordinates using the inverse equations of the spherical model. Subsequently, these points were back-projected into pinhole image coordinates using perspective projection equations.

The application of distortion-aware kernels to the model trained on our hand-labeled dataset yielded an approximate mIoU improvement of 1% on the test set across different dimensions of the

Model	Spherical conv.	Layers/block modified	% mIoU	$\Delta\%$
ERFNet	No	-	58.24	-
ERFNet	Yes	1	59.57	+1.33
HardNet	No	-	64.30	-
HardNet	Yes	I (1 st conv)	65.07	+0.77
HardNet	Yes	5	63.44	-0.86

Table 4 Quantitative results for our hand-annotated spherical dataset. $\Delta\%$ represents the mIoU difference with respect to the original architecture with regular convolutions. For details on layers/block numbers, please refer to Tables A1 and A2 in Appendix A.

training patches. However, the image degradation resulting from the transformation of patches into pinhole images led to a corresponding decline in results. Nevertheless, the relative improvement attributable to the new convolutional sampling pattern was confirmed, with a slight enhancement in free space and background generalization capabilities.

Notably, our annotated spherical dataset features few labeled classes, exhibits a high class imbalance, and consists of a limited number of examples, rendering it challenging to successfully train a semantic segmentation algorithm. Consequently, these three factors prevented the full realization of the benefits offered by our approach.

One prominent difference between the experiments conducted on the two datasets lies in the FoVs of the training and test images. For Cityscapes, the corresponding FoVs are the same, whereas for our hand-labeled datasets the training image FoVs depend on the dimensions of the crops, while the test images maintain the same FoVs as the original spherical images (180° and

160°). It is important to highlight that the performance enhancements, albeit moderate, were achieved solely by modifying the sampling location of the convolutional filters, with all other parameters unchanged.

5 Conclusion

In this work, we conducted an analysis of deep learning techniques for the purpose of semantic segmentation in urban traffic scenarios. Our objective was to implement a real-time algorithm suitable for deployment in the short-range system of a self-driving car. Specifically, we investigated a method to enhance algorithm performance in automotive scenarios while meeting the real-time requirement.

To address the limitations of fisheye images, we performed segmentation on a different representation, namely spherical images. This geometric transformation mitigates border distortion while preserving image quality. We modified the architecture of the models by replacing regular convolutions with distortion-aware convolutions. The sampling pattern of the kernels was adjusted to

Training dataset	Model	Spherical conv.	% mIoU	$\Delta\%$
pinhole Cityscapes	ERFNet	No	27.21	-
	ERFNet	Yes	27.67	+0.46
	HarDNet	No	16.08	-
	HarDNet	Yes	17.26	+1.18

Table 5 Performance improvement of models resulting from the application of spherical kernels in the first convolutional layer. The models have been trained on 512×1024 pinhole images, while the inference results are reported for the spherical Cityscapes validation set, consisting of 368×1024 images. $\Delta\%$ indicates the improvement relative to the model employing standard convolutions.

Training dataset	Model	Spherical conv.	% mIoU	$\Delta\%$
248x256 patches	HarDNet	No	55.17	-
	HarDNet	Yes	56.37	+1.20
186x192 patches	HarDNet	No	40.25	-
	HarDNet	Yes	40.87	+0.62
186x192 patches+	HarDNet	No	50.09	-
	HarDNet	Yes	51.72	+1.63
248x256 patches pin	HarDNet	No	38.59	-
	HarDNet	Yes	38.38	-0.21
186x192 patches pin	HarDNet	No	23.74	-
	HarDNet	Yes	24.08	+0.34
186x192 patches+ pin	HarDNet	No	36.52	-
	HarDNet	Yes	36.77	+0.25

Table 6 Assessment of the impact of employing spherical kernels in the first convolutional layer. The inference results are provided for the hand-annotated test set composed of 496×512 images (with original FoVs). $\Delta\%$ indicates the improvement relative to the model using standard convolutions. The set denoted as *patches* is generated by cropping *two* partially overlapped patches from the center of the original spherical images. Conversely, the *patches+* set is created by cropping *four* partially overlapping patches. *Pin* indicates the conversion to pinhole images.

encode the distortion introduced by the spherical model. We created three distinct datasets consisting of spherical images to demonstrate the benefits of our approach: geometrically transforming the Cityscapes dataset, generating synthetic images using an urban scene software simulator (CARLA), and manually annotating real spherical images. Our method improved performance across all three spherical datasets, indicating that convolutional neural networks (CNNs) employing this type of kernels can outperform baseline models with standard convolutions. The advantages of spherical convolutions can be further

exploited when dealing with a larger number of labeled classes. In our hand-labeled dataset, only two classes encompass 95% of the pixels, which makes it relatively easy to generalize even for a model with standard convolutions. By including classes that are more affected by distortion, such as fences, buildings, and vegetation, we expect a more substantial enhancement as the spherical kernels sampling pattern can be fully leveraged. This statement is supported by the results obtained from the synthetic CARLA dataset, where the percentage mIoU difference compared to the baseline is nearly five times higher than that

of the other datasets. This discrepancy is likely due to the larger number of categories and wider FoV.

We observed that CNNs trained on pinhole images and subsequently redesigned with distortion-aware convolutional layers yield improved segmentation predictions on spherical images. This finding suggests that by maintaining the same set of parameters and only altering the sampling locations of the convolutional kernels, networks that perform well on standard pinhole images can effectively infer on images obtained from a different projection model. Although the achieved quantitative differences are not substantial, networks modified in this manner can be useful in semi-automatic labeling systems. By alleviating, at least partially, the effort required to create a dataset of spherical images, this approach simplifies the labeling process and allows human annotators to focus only on regions with incorrect outputs.

It is important to note that explicitly encoding invariance to specific distortions within the neural network enables the network itself to avoid learning these distortions during the training process. Moreover, this approach can be extended and applied to various projection models, such as the cylindrical model, facilitating the transition between different types of geometric distortions.

Appendix A Networks architecture

This appendix serves as a supplementary resource for the paper, providing additional information on the specific neural network architectures used in the study. In the following, we present a brief analysis of ERFNet and HarDNet models, highlighting their essential components and functionalities.

Specifically, Table A1 presents a comprehensive description of the ERFNet architecture, emphasizing important aspects such as the number of output channels and the output resolution in the last two columns. The *Downsampler Block* is comprised of a 3x3 convolution operation followed by a max pooling operation, enabling effective feature extraction. Additionally, the *Non-Bottleneck-1D* module, described in Section 3.3, plays a crucial role in enhancing performance. Furthermore, the *Upsampler Block*, consisting of a transposed convolution (deconvolution) operation, enables the reconstruction of high-resolution feature maps from the encoded information.

Table A2 provides a comprehensive overview of the HarDNet architecture. The last two columns of the table indicate the number of output channels and the output resolution, respectively. One of the pivotal components, the *Harmonic Dense Block (HDB)*, is central to the architecture. Comprising a series of 3×3 convolutional layers, the *HDB* is organized as described in Section 3.3. Finally, the

operation denoted as *TransitionUp* involves a simple yet effective bilinear interpolation technique.

References

- [1] Penmetsa, P., Adanu, E.K., Wood, D., Wang, T., Jones, S.L.: Perceptions and expectations of autonomous vehicles – a snapshot of vulnerable road user opinion. *Technological Forecasting and Social Change* **143**, 9–13 (2019)
- [2] Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., Adam, H.: Encoder-decoder with atrous separable convolution for semantic image segmentation. In: *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 801–818 (2018)
- [3] Xie, E., Wang, W., Yu, Z., Anandkumar, A., Alvarez, J.M., Luo, P.: Segformer: Simple and efficient design for semantic segmentation with transformers. *Advances in Neural Information Processing Systems* **34**, 12077–12090 (2021)
- [4] Cheng, B., Misra, I., Schwing, A.G., Kirillov, A., Girdhar, R.: Masked-attention mask transformer for universal image segmentation. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1290–1299 (2022)
- [5] Guerrero-Viu, J., Fernandez-Labrador, C., Demonceaux, C., Guerrero, J.J.: What’s in my room? object recognition on indoor panoramic images. In: *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 567–573 (2020). IEEE
- [6] Xu, Y., Wang, K., Yang, K., Sun, D., Fu, J.: Semantic segmentation of panoramic images using a synthetic dataset. *Artificial Intelligence and Machine Learning in Defense Applications* (2019)
- [7] Orhan, S., Bastanlar, Y.: Semantic segmentation of outdoor panoramic images. *Signal, Image and Video Processing* **16**(3), 643–650 (2021)
- [8] Zhang, J., Yang, K., Ma, C., Reiss, S., Peng, K., Stiefelhagen, R.: Bending reality: Distortion-aware transformers for adapting to panoramic semantic segmentation. In: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2022)
- [9] Rueden, L., Mayer, S., Beckh, K., Georgiev, B., Giesselbach, S., Heese, R., Kirsch, B., Pfrommer, J., Pick, A., Ramamurthy, R., Walczak, M., Garcke, J., Bauckhage, C., Schuecker, J.: Informed machine learning – a taxonomy and survey of integrating prior knowledge into learning systems. IEEE

	Layer	Type	Out Chann.	Out Res.
ENCODER	1	Downsampler Block	16	(H/2)x(W/2)
	2	Downsampler Block	64	(H/4)x(W/4)
	3-7	5 x Non-bt-1D	64	(H/4)x(W/4)
	8	Downsampler Block	128	(H/8)x(W/8)
	9	Non-bt-1D (dilated 2)	128	(H/8)x(W/8)
	10	Non-bt-1D (dilated 4)	128	(H/8)x(W/8)
	11	Non-bt-1D (dilated 8)	128	(H/8)x(W/8)
	12	Non-bt-1D (dilated 16)	128	(H/8)x(W/8)
	13	Non-bt-1D (dilated 2)	128	(H/8)x(W/8)
	14	Non-bt-1D (dilated 4)	128	(H/8)x(W/8)
	15	Non-bt-1D (dilated 8)	128	(H/8)x(W/8)
	16	Non-bt-1D (dilated 16)	128	(H/8)x(W/8)
DECODER	17	Upsampler Block	64	(H/4)x(W/4)
	18-19	2 x Non-bt-1D	64	(H/4)x(W/4)
	20	Upsampler Block	16	(H/2)x(W/2)
	21-22	2 x Non-bt-1D	16	(H/2)x(W/2)
	23	Upsampler Block	C	HxW

Table A1 ERFNet architecture. The last two columns show the number of output channels and the output resolution, respectively. The *Downsampler Block* comprises a 3x3 convolution followed by a max pooling operation. *Non-bt-1D* refers to the newly introduced non-bottleneck-1D module. The *Upsampler Block* consists of a transposed convolution (deconvolution) operation. *C* represents the number of classes considered.

- Transactions on Knowledge and Data Engineering **35**(1), 614–633 (2023)
- [10] Rath, M., Condurache, A.P.: Boosting deep neural networks with geometrical prior knowledge: A survey (2023)
- [11] Dai, J., Qi, H., Xiong, Y., Li, Y., Zhang, G., Hu, H., Wei, Y.: Deformable convolutional networks. In: Proceedings of the IEEE International Conference on Computer Vision, pp. 764–773 (2017)
- [12] Zhu, X., Hu, H., Lin, S., Dai, J.: Deformable convnets v2: More deformable, better results. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 9308–9316 (2019)
- [13] Deng, L., Yang, M., Li, H., Li, T., Hu, B., Wang, C.: Restricted deformable convolution-based road scene semantic segmentation using surround view cameras. IEEE Transactions on Intelligent Transportation Systems, 1–13 (2019)
- [14] Su, Y.-C., Grauman, K.: Learning spherical convolution for fast features from 360 imagery. Advances in Neural Information Processing Systems **30** (2017)
- [15] Su, Y.-C., Grauman, K.: Kernel transformer networks for compact spherical convolution. 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) (2019)

	Block	Type	Out Chann.	Out Res.
ENCODER	I	3x3 Conv	16	(H/2)x(W/2)
		3x3 Conv	24	(H/2)x(W/2)
		3x3 Conv	32	(H/4)x(W/4)
		3x3 Conv	48	(H/4)x(W/4)
	1	HDB (s=4)	48	(H/4)x(W/4)
		1x1 Conv	64	(H/4)x(W/4)
		Avg pooling	64	(H/8)x(W/8)
		HDB (s=4)	78	(H/8)x(W/8)
	2	1x1 Conv	96	(H/8)x(W/8)
		Avg pooling	96	(H/16)x(W/16)
		HDB (s=8)	160	(H/16)x(W/16)
		1x1 Conv	160	(H/16)x(W/16)
	3	Avg pooling	160	(H/32)x(W/32)
		HDB (s=8)	214	(H/32)x(W/32)
		1x1 Conv	224	(H/32)x(W/32)
		Avg pooling	224	(H/64)x(W/64)
4	HDB (s=8)	286	(H/64)x(W/64)	
	1x1 Conv	320	(H/64)x(W/64)	
DECODER	6	TransitionUp	320	(H/32)x(W/32)
		1x1 Conv	267	(H/32)x(W/32)
		HDB (s=8)	374	(H/32)x(W/32)
	7	TransitionUp	374	(H/16)x(W/16)
		1x1 Conv	187	(H/16)x(W/16)
		HDB (s=8)	238	(H/16)x(W/16)
	8	TransitionUp	238	(H/8)x(W/8)
		1x1 Conv	119	(H/8)x(W/8)
		HDB (s=4)	126	(H/8)x(W/8)
	9	TransitionUp	126	(H/4)x(W/4)
		1x1 Conv	63	(H/4)x(W/4)
		HDB (s=4)	48	(H/4)x(W/4)
	F	Final 3x3 Conv	C	(H/4)x(W/4)
		TransitionUp	C	HxW

Table A2 HarDNet architecture. The number of output channels and the output resolution are indicated in the last two columns. *HDB* is the *Harmonic Dense Block*. The *TransitionUp* operation refers to a simple bilinear interpolation.

[16] Su, Y.-C., Grauman, K.: Learning spherical convolution for 360 recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1–1 (2021)

[17] Cohen, T.S., Geiger, M., Köhler, J., Welling, M.: Spherical CNNs. In: *International Conference on Learning Representations* (2018)

[18] Yu, D., Ji, S.: Grid based spherical cnn for object detection from panoramic images. *Sensors* **19**(11), 2622 (2019)

[19] Lee, Y., Jeong, J., Yun, J., Cho, W., Yoon, K.-J.: SpherePHD: Applying CNNs on a spherical PolyHeDron representation of 360° images. In: *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition*

- (CVPR) (2019) 1123
- [20] Jiang, C.M., Huang, J., Kashinath, K., Prabhath, Marcus, P., Niessner, M.: Spherical CNNs on unstructured grids. In: International Conference on Learning Representations (2019) 1124
- [21] Zhao, F., Xia, S., Wu, Z., Duan, D., Wang, L., Lin, W., Gilmore, J.H., Shen, D., Li, G.: Spherical u-net on cortical surfaces: Methods and applications. *Information Processing in Medical Imaging*, 855–866 (2019) 1125
- [22] Zhang, C., Liwicki, S., Smith, W., Cipolla, R.: Orientation-aware semantic segmentation on icosahedron spheres. *2019 IEEE/CVF International Conference on Computer Vision (ICCV)* (2019) 1126
- [23] Eder, M., Shvets, M., Lim, J., Frahm, J.-M.: Tangent Images for Mitigating Spherical Distortion (2019) 1127
- [24] Eder, M., Frahm, J.-M.: Convolutions on spherical images. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, pp. 1–5 (2019) 1128
- [25] Boomsma, W., Frellsen, J.: Spherical convolutions and their application in molecular modelling. In: *Advances in Neural Information Processing Systems*, pp. 3433–3443 (2017) 1129
- [26] Coors, B., Paul-Condurache, A., Geiger, A.: Spherenet: Learning spherical representations for detection and classification in omnidirectional images. In: *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 518–533 (2018) 1130
- [27] Tateno, K., Navab, N., Tombari, F.: Distortion-aware convolutional filters for dense prediction in panoramic images. In: *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 707–722 (2018) 1131
- [28] Fernandez-Labrador, C., Facil, J.M., Perez-Yus, A., Demonceaux, C., Civera, J., Guerrero, J.J.: Corners for layout: End-to-end layout recovery from 360 images. *IEEE Robotics and Automation Letters* **5**(2), 1255–1262 (2020) 1132
- [29] Shakerinava, M., Ravanbakhsh, S.: Equivariant networks for pixelized spheres. In: Meila, M., Zhang, T. (eds.) *Proceedings of the 38th International Conference on Machine Learning. Proceedings of Machine Learning Research*, vol. 139, pp. 9477–9488 (2021) 1133
- [30] Jiang, W., Wu, Y., Guan, L., Zhao, J.: Dfnet: Semantic segmentation on panoramic images with dynamic loss weights and residual fusion 1134

- block. In: 2019 International Conference on Robotics and Automation (ICRA), pp. 5887–5892 (2019)
- [31] Yang, K., Hu, X., Bergasa, L.M., Romera, E., Wang, K.: Pass: Panoramic annular semantic segmentation. *IEEE Transactions on Intelligent Transportation Systems*, 1–15 (2019)
- [32] Ao, Y., Wang, J., Zhou, M., Lindenberg, R., Yang, M.Y.: Fully convolutional networks for street furniture identification in panorama images. *ISPRS - International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences* **XLII-2/W13**, 13–20 (2019)
- [33] Zhang, C., He, S., Liwicki, S.: A spherical approach to planar semantic segmentation. In: 2020 British Machine Vision Conference (BMVC)
- [34] Yang, K., Hu, X., Stiefel, R.: Is context-aware CNN ready for the surroundings? panoramic semantic segmentation in the wild. *IEEE Transactions on Image Processing* **30**, 1866–1881 (2021)
- [35] Liu, Y., Wang, Y., Du, H., Cai, S.: Spherical transformer: Adapting spherical signal to convolutional networks. In: *Pattern Recognition and Computer Vision*, pp. 15–27 (2022)
- [36] Hu, X., An, Y., Shao, C., Hu, H.: Distortion convolution module for semantic segmentation of panoramic images based on the image-forming principle. *IEEE Transactions on Instrumentation and Measurement* **71**, 1–12 (2022)
- [37] Jaus, A., Yang, K., Stiefel, R.: Panoramic panoptic segmentation: Towards complete surrounding understanding via unsupervised contrastive learning. In: 2021 IEEE Intelligent Vehicles Symposium (IV) (2021)
- [38] Ma, C., Zhang, J., Yang, K., Roitberg, A., Stiefel, R.: DensePASS: Dense panoramic semantic segmentation via unsupervised domain adaptation with attention-augmented context exchange. In: 2021 IEEE International Intelligent Transportation Systems Conference (ITSC) (2021)
- [39] Cordts, M., Omran, M., Ramos, S., Rehfeld, T., Enzweiler, M., Benenson, R., Franke, U., Roth, S., Schiele, B.: The cityscapes dataset for semantic urban scene understanding. In: *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016)
- [40] Geiger, A., Lenz, P., Urtasun, R.: Are we ready for autonomous driving? the KITTI

- vision benchmark suite. In: 2012 IEEE Conference on Computer Vision and Pattern Recognition (2012)
- [41] Sekkat, A.R., Dupuis, Y., Vasseur, P., Honeine, P.: The OmniScape dataset. In: 2020 IEEE International Conference on Robotics and Automation (ICRA) (2020)
- [42] Deng, L., Yang, M., Qian, Y., Wang, C., Wang, B.: Cnn based semantic segmentation for urban traffic scenes using fisheye camera. In: 2017 IEEE Intelligent Vehicles Symposium (IV), pp. 231–236 (2017)
- [43] Sáez, A., Bergasa, L.M., Romeral, E., López, E., Barea, R., Sanz, R.: Cnn-based fisheye image real-time semantic segmentation. In: 2018 IEEE Intelligent Vehicles Symposium (IV), pp. 1039–1044 (2018). IEEE
- [44] Shah, S., Dey, D., Lovett, C., Kapoor, A.: Airsim: High-fidelity visual and physical simulation for autonomous vehicles. Springer Proceedings in Advanced Robotics, 621–635 (2017)
- [45] Rong, G., Shin, B.H., Tabatabaee, H., Lu, Q., Lemke, S., Možeiko, M., Boise, E., Uhm, G., Gerow, M., Mehta, S., Agafonov, E., Kim, T.H., Sterner, E., Ushiroda, K., Reyes, M., Zelenkovsky, D., Kim, S.: LGSVL Simulator: A High Fidelity Simulator for Autonomous Driving (2020)
- [46] <http://rfpro.com>. Accessed: 2023-06-18
- [47] Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., Koltun, V.: CARLA: An open urban driving simulator. In: Proceedings of the 1st Annual Conference on Robot Learning, pp. 1–16 (2017)
- [48] Romera, E., Alvarez, J.M., Bergasa, L.M., Arroyo, R.: Erfnet: Efficient residual factorized convnet for real-time semantic segmentation. IEEE Transactions on Intelligent Transportation Systems **19**(1), 263–272 (2017)
- [49] Chao, P., Kao, C.-Y., Ruan, Y.-S., Huang, C.-H., Lin, Y.-L.: Hardnet: A low memory traffic network. In: Proceedings of the IEEE International Conference on Computer Vision, pp. 3552–3561 (2019)
- [50] Chen, L.-C., Zhu, Y., Papandreou, G., Schroff, F., Adam, H.: Encoder-decoder with atrous separable convolution for semantic image segmentation. In: Proceedings of the European Conference on Computer Vision (ECCV), pp. 801–818 (2018)
- [51] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1225–1275

- 1276 770–778 (2016)
1277
1278 [52] Alvarez, J., Petersson, L.: Decomposeme:
1279 Simplifying convnets for end-to-end learning.
1280 arXiv preprint (2016)
1281
1282
1283
1284 [53] Long, J., Shelhamer, E., Darrell, T.: Fully
1285 convolutional networks for semantic segmen-
1286 tation. In: Proceedings of the IEEE Con-
1287 ference on Computer Vision and Pattern
1288 Recognition, pp. 3431–3440 (2015)
1289
1290
1291
1292
1293 [54] Ronneberger, O., Fischer, P., Brox, T.: U-
1294 net: Convolutional networks for biomedical
1295 image segmentation. In: International Con-
1296 ference on Medical Image Computing and
1297 Computer-assisted Intervention, pp. 234–241
1298 (2015). Springer
1299
1300
1301
1302
1303
1304 [55] Badrinarayanan, V., Kendall, A., Cipolla,
1305 R.: Segnet: A deep convolutional encoder-
1306 decoder architecture for image segmenta-
1307 tion. IEEE transactions on pattern analysis
1308 and machine intelligence **39**(12), 2481–2495
1309 (2017)
1310
1311
1312
1313
1314
1315 [56] Paszke, A., Chaurasia, A., Kim, S., Culur-
1316 ciello, E.: Enet: A deep neural network archi-
1317 tecture for real-time semantic segmentation.
1318 arXiv preprint (2016)
1319
1320
1321
1322 [57] Huang, G., Liu, Z., Van Der Maaten, L.,
1323 Weinberger, K.Q.: Densely connected con-
1324 volutional networks. In: Proceedings of the
1325
1326 IEEE Conference on Computer Vision and
Pattern Recognition, pp. 4700–4708 (2017)
[58] Kingma, D., Ba, J.: Adam: A method for
stochastic optimization. International Con-
ference on Learning Representations (2014)

Supplementary Files

This is a list of supplementary files associated with this preprint. Click to download.

- [ESM1.mp4](#)